

```
enter the no.of subjects: 5
enter 1 the subect mark90
enter 2 the subect mark80
enter 3 the subect mark100
enter 4 the subect mark100
enter 5 the subect mark60
```

```
# def intro(*args):
#     user_name = input("enter your name: ")
#     total_subject = int(input("enter the no.of subjects: ")) #5
#     subject_marks = []
#     count = 1
#     for i in range(total_subject):
#         subject_marks.append(int(input(f"enter the {count} subject mark: ")))
#         count +=1
#     print("the entered marks are :", subject_marks)

# intro()

# def total_marks(*args):
#     total_mark = 0
#     for i in subject_marks:
#         total_mark = total_mark + i
#     print("total marks", total_mark)

# total_marks()

# def avg_mark(*args):
#     avg = total_mark/total_subject
#     print("avg marks",avg)

# avg_mark()

# s = [17,99,68,94,100] # highest and lowest ?
# highest = 100
# for i in s:
#     if i < highest:
#         highest = i

# print(highest)

class and object
```

```
File "/tmp/ipykernel_1708/2614285903.py", line 40
    class and object .
      ^
SyntaxError: invalid syntax
```

Project 1: Student Report Card System

Problem Statement

A school wants to automate the process of generating student report cards.

Your task is to build a program that allows the user to enter a student's details and subject marks, then generates a complete report card.

Requirements

- Ask for the student's name.
- Ask how many subjects the student has.
- Collect the marks for each subject.
- Display all the entered marks.
- Calculate the total marks.
- Calculate the average marks.
- Display the highest and lowest marks.
- Display whether the student has **Passed** or **Failed**.
- Assign a grade based on the average marks.
- Print a well-formatted report card.

Bonus Challenge

- Display the subject number along with each mark.
- Prevent the user from entering marks below 0 or above 100.

```
=====
Student Name : John

Marks       : [90,100,99,88,75]

Total Marks : 452

Average     : 90.4

Highest     : 100

Lowest      : 75

Result      : PASS
=====
```

Start coding or generate with AI.

```
def report_card(*args):
    name = input("enter student name: ")
    total_subject = int(input("how many subject the student has ? "))
    marks = []
    for i in range(total_subject):
        marks.append(int(input("enter the subject mark")))
    print("The marks are given in the list: ", marks)

    total_marks = 0
    for i in marks:
        total_marks+=i
    avg_marks = total_marks/total_subject
    highest = 0
    for i in marks:
        if i > highest:
            highest = i
    lowest = marks[0]
    for i in marks:
        if i < lowest:
            lowest = i
    print(f"the hisghest mark is {highest} and the lowest mark is {lowest}")

    if total_marks >120:
        output = "he has passed the test"
    else:
        output = "he has faileasd the test"

    print(output)

report_card()
```

```
enter student name: ganesh
how many subject the student has ? 3
enter the subject mark100
enter the subject mark50
enter the subject mark100
The marks are given in the list: [100, 50, 100]
the hisghest mark is 100 and the lowest mark is 50
he has faileds the test
```

Project 2: ATM Banking System

Problem Statement

A bank wants to develop a simple ATM simulation.

Your task is to create a menu-driven program that allows users to perform basic banking operations.

Requirements

The ATM should display the following options:

1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transaction History
5. Exit



The program should:

- Start with an initial account balance.
- Allow the user to deposit money.
- Allow the user to withdraw money.
- Prevent withdrawal if there is not enough balance.
- Display the current balance whenever requested.
- Continue showing the menu until the user chooses Exit.
- Display a thank-you message before closing.

Bonus Challenge

- Store every transaction and display the complete transaction history before exiting.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Project 3: Quiz Application

Problem Statement

A teacher wants a simple quiz application for students.

Your task is to build a quiz system that asks multiple questions and calculates the student's final score.

Requirements

- Ask the user to enter their name.
- Display one question at a time.
- Accept the user's answer.
- Check whether the answer is correct.
- Display whether the answer is correct or incorrect.
- Keep track of the score.
- After all questions are completed, display:
 - Student Name
 - Total Questions
 - Correct Answers
 - Wrong Answers
 - Percentage
 - Grade
- Display a final message based on the student's performance.

Bonus Challenge

- Allow the user to play the quiz again without restarting the program.
- Display the correct answer whenever the student answers incorrectly.

